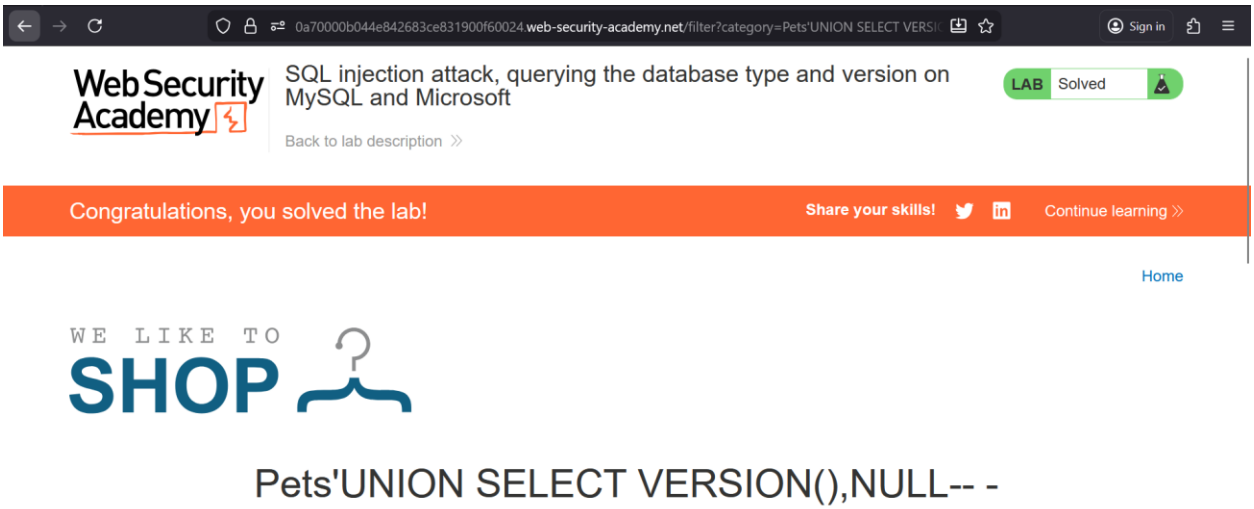
Title: determined the website that have SQL injection vulnerability that get the version database

SQL Injection forces a web application to execute unintended SQL commands by injecting malicious input into queries. Failures typically lead to unauthorized access to the database, exposure of sensitive data, deletion or modification of records, or full compromise of the backend system.

Reproduction Steps/Point of view:

1.visit **https://0a70000b044e842683ce831900f60024.web-security-academy.net/**

2. perform a malicious input to get the version of the database get into search Pet and write('UNION SELECT VESRION(),NULL-- -) like this
https://0a70000b044e842683ce831900f60024.web-security-academy.net/filter?category=Pets%27UNION%20SELECT%20VERSION(),NULL--%20-



Impact statement:

Exploitation of this vulnerability allows an unauthenticated attacker to access sensitive version of that database including names, emails, and payment information, potentially affecting all registered users. This poses a critical risk to customer privacy, regulatory compliance, and organizational reputation.

Mitigation:

To remediate this SQL Injection vulnerability exploited via a UNION-based attack, it is strongly recommended to implement **Parameterized Queries (Prepared Statements)** for all database queries involving user-supplied input such as the `category` parameter. This ensures that user input cannot be appended or manipulated to inject additional SQL clauses like `UNION SELECT`.

Additionally, enforce **strict input validation and whitelisting** on the `category` parameter, accepting only predefined expected values and rejecting any input containing SQL keywords or special characters such as `'`, `UNION`, `SELECT`, or `--`.

Apply the **principle of least privilege** to the database account used by the application, ensuring it has no access to sensitive system views such as `version` or any internal **MySQL and Microsoft** metadata tables. This limits the damage an attacker can cause even if injection is partially successful.

Ensure that **detailed database error messages and version information** are never exposed to the end user, as disclosing the database type and version (e.g., Oracle) significantly assists attackers in tailoring further exploitation attempts.

Implement a **Web Application Firewall (WAF)** as an additional defensive layer to detect and block common UNION-based SQL Injection payloads before they reach the application.